a) Tema (Domínio)

Tema: Plataforma Web de Desenvolvimento Multiplataforma Baseada em Geração de Código (CROSSPLAT IDE).

O projeto visa solucionar o problema de usabilidade e acessibilidade de ferramentas de geração de código baseadas em Linguagens de Domínio Específico (DSLs) e no framework Xtext. O objetivo é remover a forte dependência do ambiente Eclipse IDE, democratizando o uso da abordagem multiplataforma em um ambiente 100% web, voltado para estudantes, pesquisadores e desenvolvedores.

b) Gestão e Processo

- **Metodologia:** Ágil, utilizando **Kanban** para o fluxo de desenvolvimento contínuo e gestão visual de tarefas.
- **Ferramenta:** Github Projects ou Trello para o board do Kanban.
- **Backlog Inicial (Macro):**
    - Módulo de Autenticação (Login/Cadastro)
    - Dashboard de Gestão de Projetos
    - Editor de código Web Integrado (Monaco Editor)
    - Marketplace de Modelos de Plataforma
    - Integração Backend-Gerador (Execução de JAR via linha de comando)

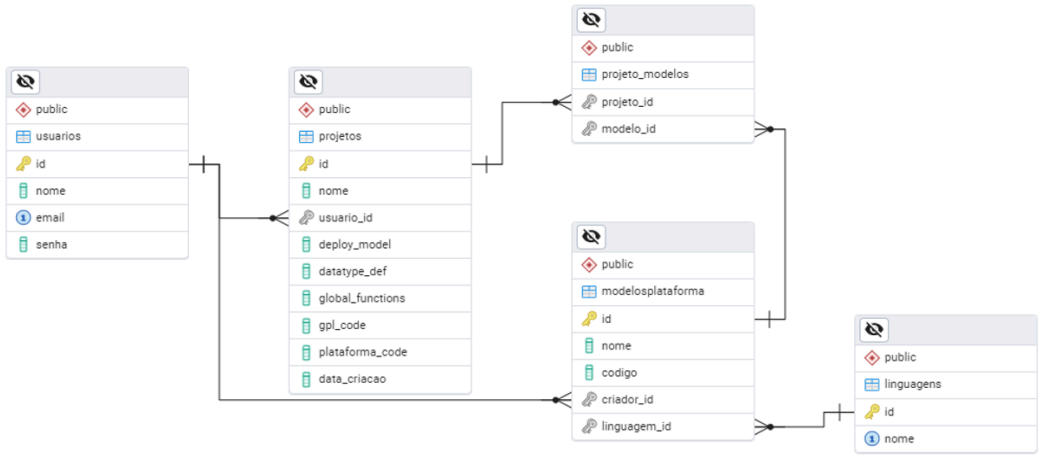
c) Identidade Visual (Style Guide)

- **Tema:** Dark Mode (foco no conforto visual para desenvolvedores).
- **Cores Principais:**
    - Fundo Principal: #121212
    - Fundo Secundário (Paineis, Sidebars): #181818 / #1e1e1e
    - Acentos (Ações primárias): #60a5fa (Azul Claro)
    - Sucesso/Output: #4ade80 (Verde)
- **Tipografia:**
    - Sistema: Inter ou Roboto.
    - Editor de Código: Fira Code ou Consolas (Monospace).
- **Ícones:** Material Symbols Outlined.

d) Modelagem do Sistema

- **Prototipação:** Estruturada via código direto (HTML/CSS) simulando o fluxo de navegação como uma Single Page Application (SPA).
- **Modelo de Banco de Dados (MER Relacional):**
    - usuarios (id, nome, email, senha)
    - projetos (id, nome, usuario_id, dados_dos_arquivos, data_criacao)
    - modelosPlataforma (id, nome, codigo, criador_id, linguagem_id)

    - projeto_modelos (projeto_id, modelo_id)
    - linguagens (id, nome)



- **Padrão Tecnológico (Stacks separadas):**
    - **Frontend:** HTML5, CSS3, REACT, Monaco Editor.
    - **Backend:** Python 3 + Flask (API RESTful) + Execução de Subprocessos Java (.JAR).
    - **Banco de Dados:** PostgreSQL.

e) Requisitos de Qualidade (NFRs)

(Detalhado no Documento de Requisitos, mas em resumo)

- **Acessibilidade:** Sistema utilizável exclusivamente via navegador web, sem exigir instalações locais.
- **Desempenho:** A compilação e retorno do código gerado devem ocorrer em menos de 5 segundos.
- **Usabilidade:** O editor de código deve oferecer uma experiência semelhante a IDEs de desktop (highlighting, numeração de linhas, proteção de blocos não editáveis).

f) Arquitetura da Aplicação

- **Padrão API REST (Client-Server / Stacks Separadas):** O Frontend se comunicará com o Backend exclusivamente via requisições HTTP (JSON).
- **Design no Backend (Adaptado de Clean Architecture / MVC):**
    - **Controllers (Rotas):** Recebem as requisições do Flask e repassam para os serviços.
    - **Services (Regras de Negócio):** Lógica de vinculação de marketplace, manipulação de arquivos temporários e chamadas ao gerador.
    - **Repositories (Acesso a Dados):** Consultas ao PostgreSQL.
    - **Core (Gerador):** A camada mais interna aciona o arquivo .jar do Xtext responsável pela conversão do código GPL para outras linguagens.

g) Arquitetura de Infraestrutura

Containerização (Docker)

A infraestrutura do projeto será baseada na **Containerização** utilizando Docker. A aplicação backend será empacotada em um container configurado com uma imagem base capaz de suportar múltiplos ambientes de execução.

Essa abordagem foi escolhida para resolver o principal requisito técnico do sistema: o backend precisa de Python para rodar a API Web (Flask) e de Java (JRE - *Java Runtime Environment*) para executar o arquivo executável (.jar) do gerador Xtext.

O container garantirá o isolamento da aplicação e proverá internamente tanto o Python quanto o JRE, eliminando problemas de incompatibilidade entre ambientes (o famoso "na minha máquina funciona"). Esse empacotamento em um único contêiner autossuficiente também permitirá que a solução seja facilmente implantada e hospedada em serviços de nuvem PaaS (*Platform as a Service*), garantindo portabilidade, estabilidade e consistência do ambiente de desenvolvimento até a produção.

REDIS-

Para resolver o problema de requisições SQL sempre que clicar em “compilar”.
Sempre que clicar em compilar, como os modelos de plataforma raramente vão mudar, utilizar o cache para poupar tempo.